

Design Requirements

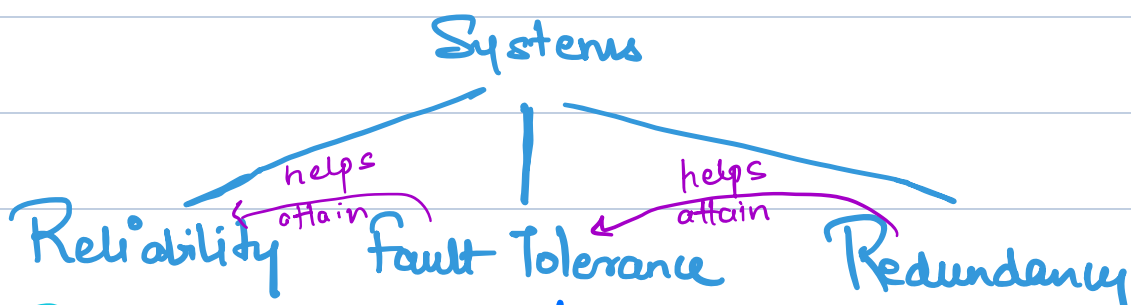
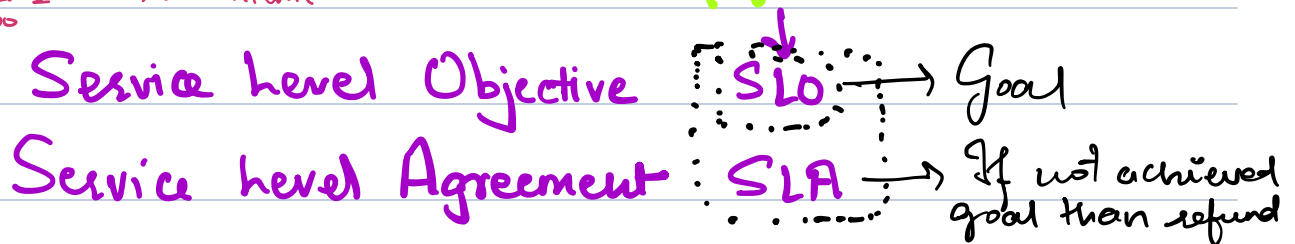
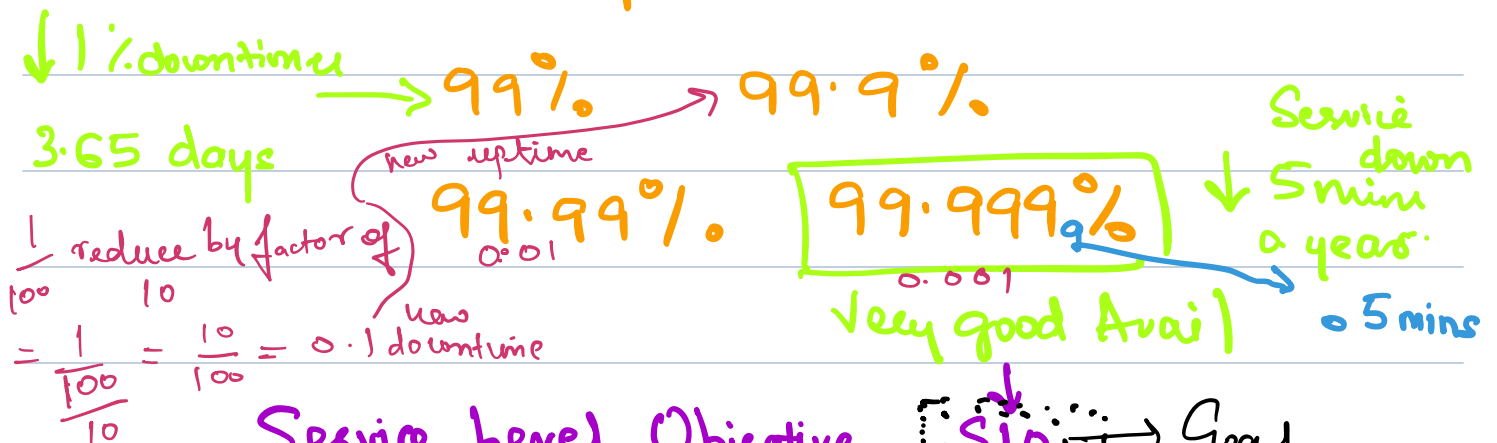
- 1) Move Data
- 2) Store Data
- 3) Transform Data

* Measure trade offs for good design

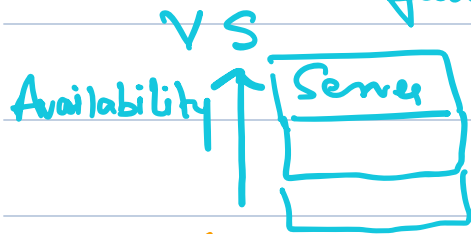
(for Service / App) Availability = $\frac{\text{uptime}}{\text{uptime} + \text{Downtime}}$

Eg for server: $\frac{23 \text{ hours}}{24 \text{ hours}} = 96\%$ 100% Availability of server is tough

⊗ Availability is measured in terms of 9s (SLO)

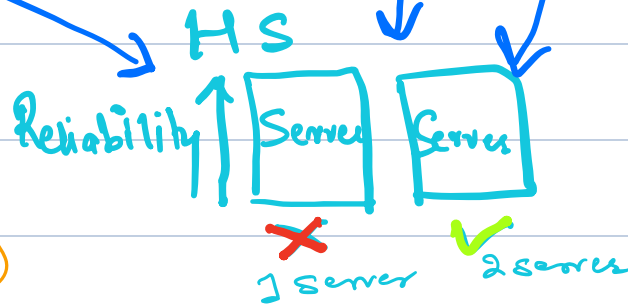
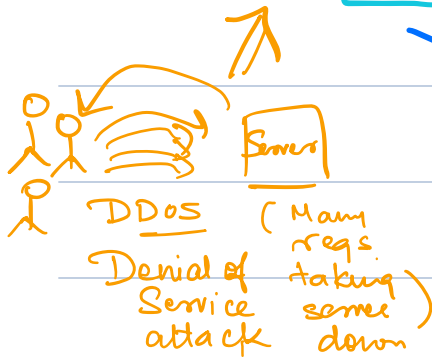


* Probability that system won't fail



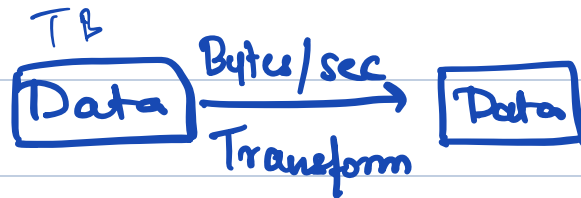
So increase of 1 server/sys goes down the other server can work meaning our entire system is fault tolerant

So we make redundant copies of server so that it can help us in case if any fault arises in 1 system



Through Put

(The amount of operations/data we can handle over a period of time)



Vertical Scaling can lead to **Single Point Failure**

Horizontal Scaling helps but it increases the complexity

Latency

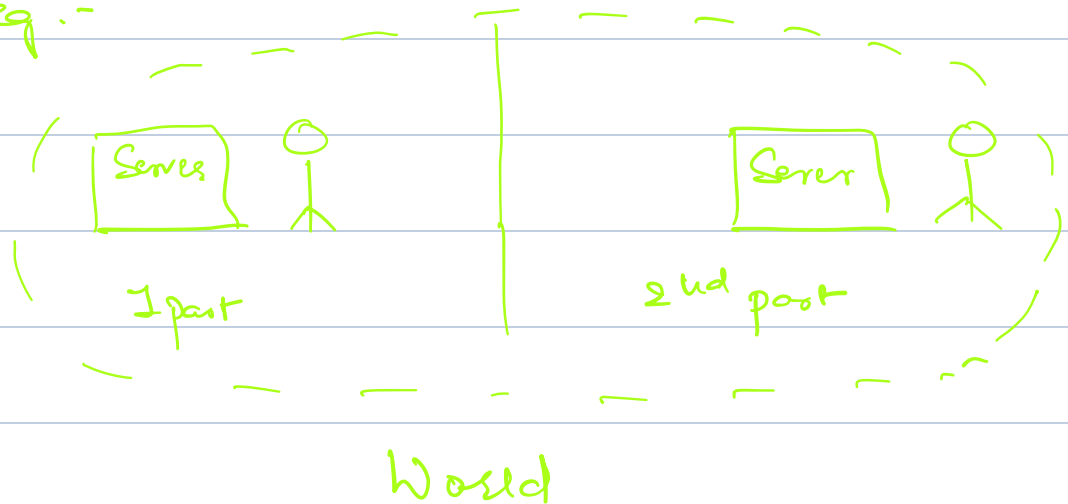


Amount of time it takes for an operation to complete

* How long does each individual req takes to complete

* If we can reduce the latency of the individual operation we can increase the overall throughput

Eg:-



↑ Availability
Reliability
Throughput ↓ Latency (CDN - Content Delivery Network)